

Meeting – June 3

Jon Earle

This Week

- Since our previous meeting, I have been focused on testing the range of the LILYGO T-Beam V1.1 with Meshtastic firmware on various settings.
- Preparing for the test was more difficult than anticipated. The GPS module on the T-Beam not only took a long time to integrate with my current script but it also proved to be much less accurate than anticipated.
- After finding an alternative solution to track where packets were sent, testing was able to begin. For different LoRa modem presets, I would bike various routes around the city, observing the changes in signal reception and quality.

Hurdles to Overcome

- The Meshtastic API allows for the sending of POSITION_APP packets which store GPS coordinates. Initially, I thought I could automate the sending of these and simply monitor the RSSI/SNR.
- However, these packets oddly did not contain GPS information. I then had to turn to using Meshtastic's automatic location broadcasts.
- By default, nodes will broadcast and store their location data every 15 minutes. By changing this to 10 seconds, the nodes would force themselves to update their positioning often enough that the location stored in the node DB would be about the same as where the packet was sent from.

Hurdles cont.

- The previous solution seemed to work as GPS and signal data were coming in consistently. However, the saved coordinates would barely ever change and would always be somewhere in Halifax I was not.
 - This was an indication that the GPS module on the T-Beam was reliable enough for this test.
- The only reliable alternative was to use my phone's GPS data to monitor where packets were being sent from.
- Now, I use an iOS app Called OwnTracks to ping my location every 5 seconds, forward it to a web server via HTTP, and then store the data on my laptop.
 - The location is updated often enough that where the packet is sent is approximately the last stored location.
 - After multiple trials, I discovered that the data collection eventually stop due to my phone hotspot disconnecting automatically. This was resolved by tethering the phone to a laptop and running a script that sent a ping request every 15 seconds to maintain the connection.

Process

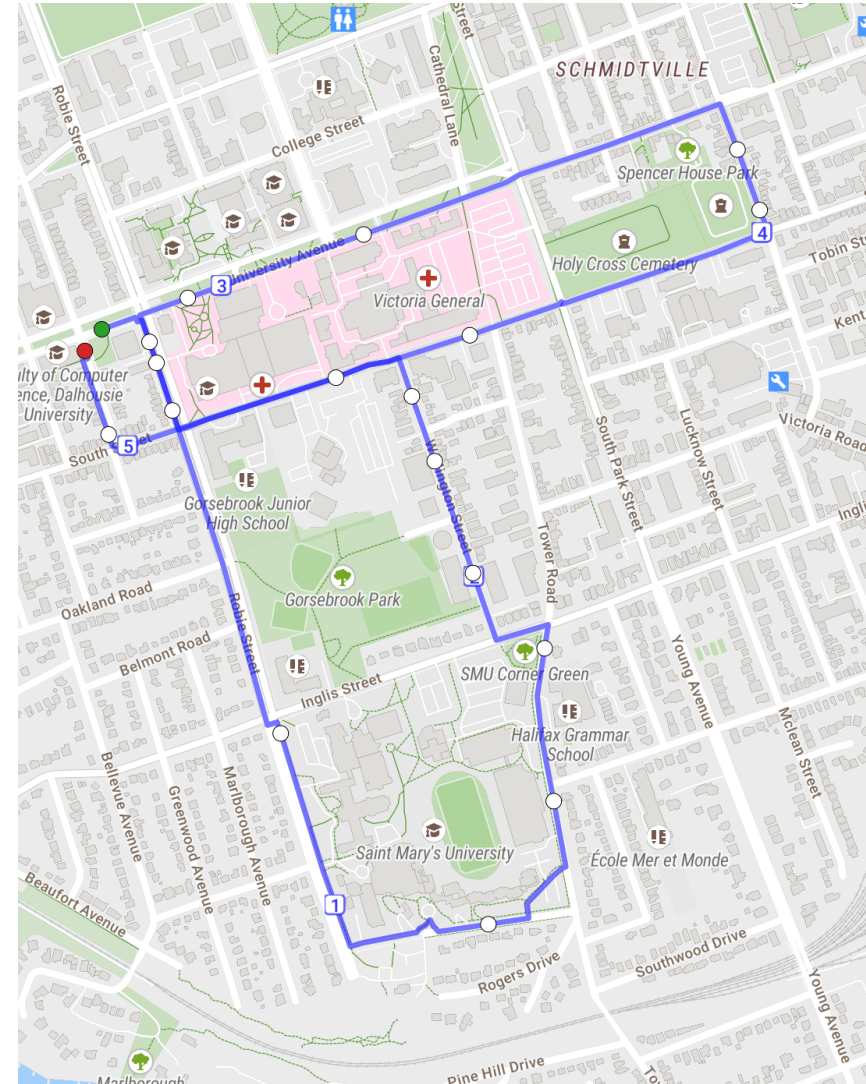
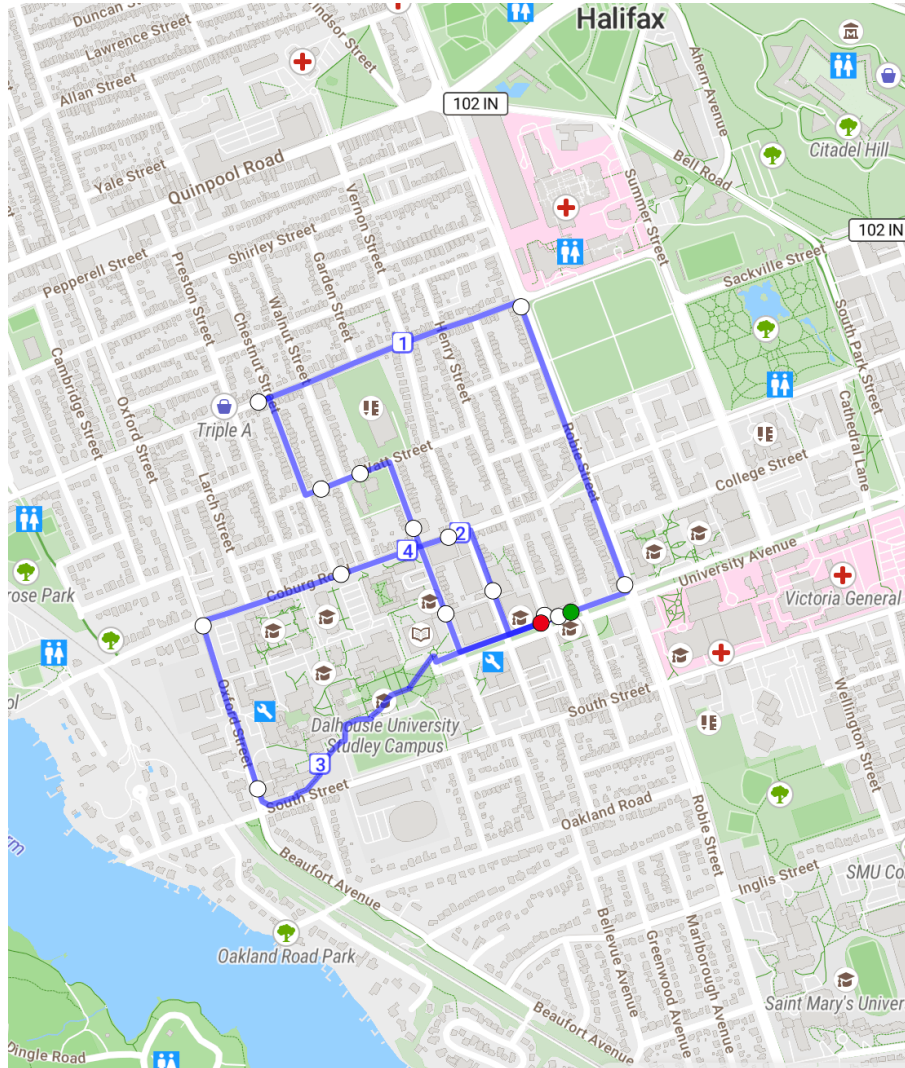
- Connect Laptop 1 to Meshtastic transceiver 1 (used serial).
- Place device next to a window, facing the travel path.
- Run `receive_position.py` on Laptop 1 to accept packets (saves to CSV)
- Connect iPhone to Laptop 2 via USB and tether the devices.
- Run `hotspot.sh` to send pings every 15 sec. (keeps hotspot alive)
- Run `server.py` to start Flask server and check that OwnTracks is connected to server via HTTP and sending fresh packets.
- Connect Laptop 2 to the Meshtastic transceiver 2 (elevated on pole)
- Run `send_position.py` on Laptop 2 to send packets
- Fix Meshtastic transceiver 2 to bag and begin biking.
- If everything works, analyze data and repeat.

Test Results

- My first tests used the SHORT_FAST setting and biking within a 600m radius of the Goldberg.
 - The receiver node was only on the first floor, next to a large glass window.
- On these tests, I seriously overestimated the range of the device. Around 150m away from the Goldberg, I would lose the connection.
- It seems that the maximum observed range of around 175m would only hold when I was on University Ave., Henry St. or Edward St. These streets are either adjacent to or across from the Goldberg, thus having a stronger line-of-sight from the building

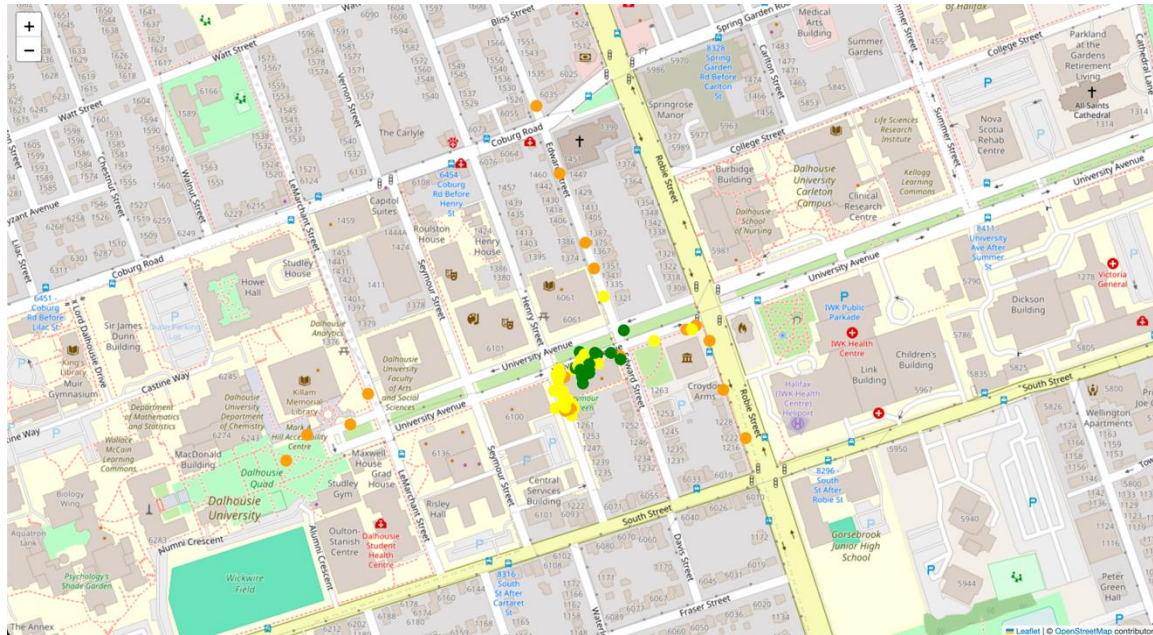
SHORT_TURBO Routes

(Majorly overestimated how far to bike. Goes to show the difference between simulation and real life)



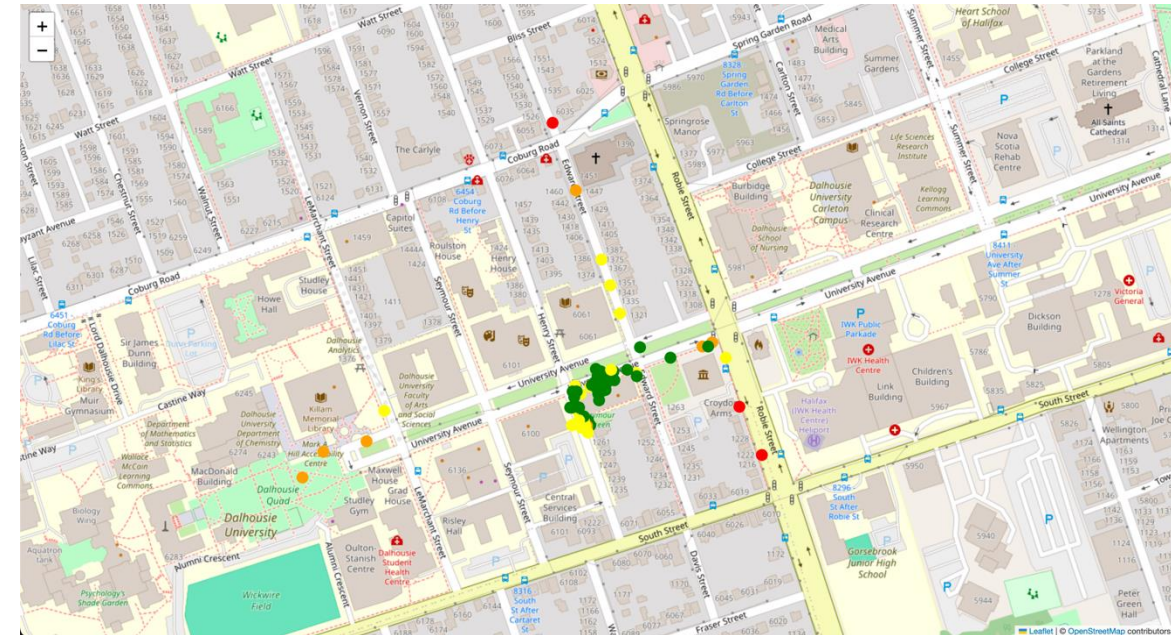
SHORT_TURBO

RSSI



- $\text{RSSI} > -90 \text{ dBm}$ = Green
- $-110 \text{ dBm} < \text{RSSI} \leq -90 \text{ dBm}$ = Yellow
- $-130 \text{ dBm} < \text{RSSI} \leq -110 \text{ dBm}$ = Orange
- $\text{RSSI} \leq -130 \text{ dBm}$ = Red

SNR



- $\text{SNR} > 10 \text{ dB}$ = Dark Green
- $5 \text{ dB} < \text{SNR} \leq 10 \text{ dB}$ = Green
- $0 \text{ dB} < \text{SNR} \leq 5 \text{ dB}$ = Yellow
- $-5 \text{ dB} < \text{SNR} \leq 0 \text{ dB}$ = Orange
- $-10 \text{ dB} < \text{SNR} \leq -5 \text{ dB}$ = Red
- $\text{SNR} \leq -10 \text{ dB}$ = Dark Red

SHORT_TURBO Approximate Range

Down an adjacent street (~350m)



With buildings (~175m)



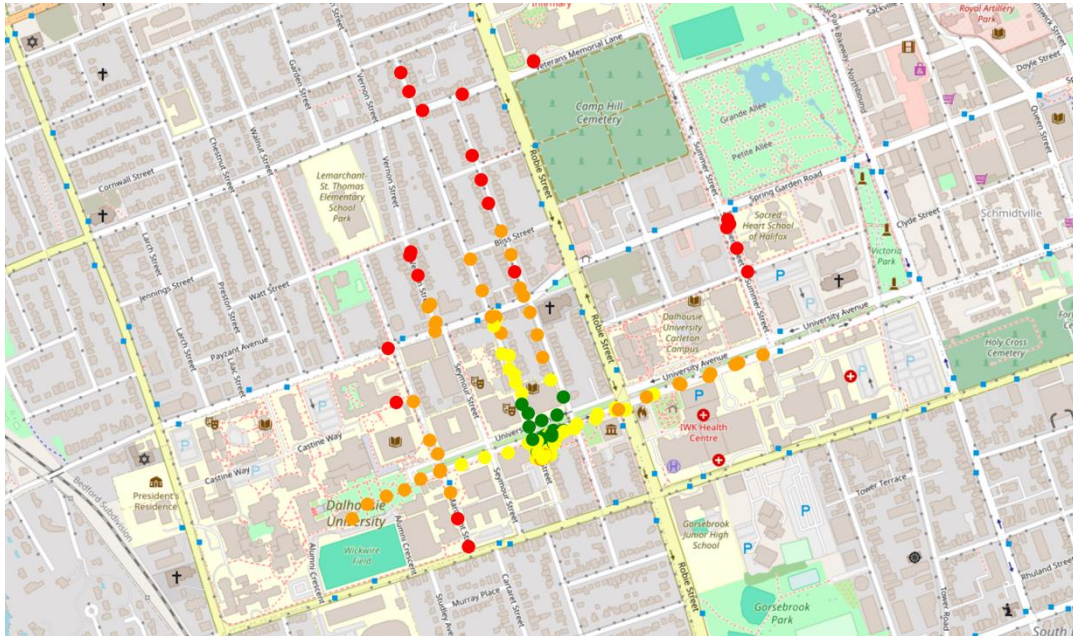
Test Results cont.

- Later tests on the LONG_FAST mode yielded similar results.
- With the LONG_FAST tests, I began biking almost 2km away from the Goldberg building. As I discovered this distance was far too great, I shortened my routes to be within ~750m of the Goldberg.
- Depending on which direction I was biking, I would leave the receiver node either in the staircase next or in Meeting Room 211, both next to large windows and on the second floor.
- As expected, packets were received at greater range and beyond more buildings. Just like SHORT_TURBO, packets were sent exceptionally further when travelling down a street adjacent to the Goldberg, especially University Ave.

LONG_FAST (Room 211)

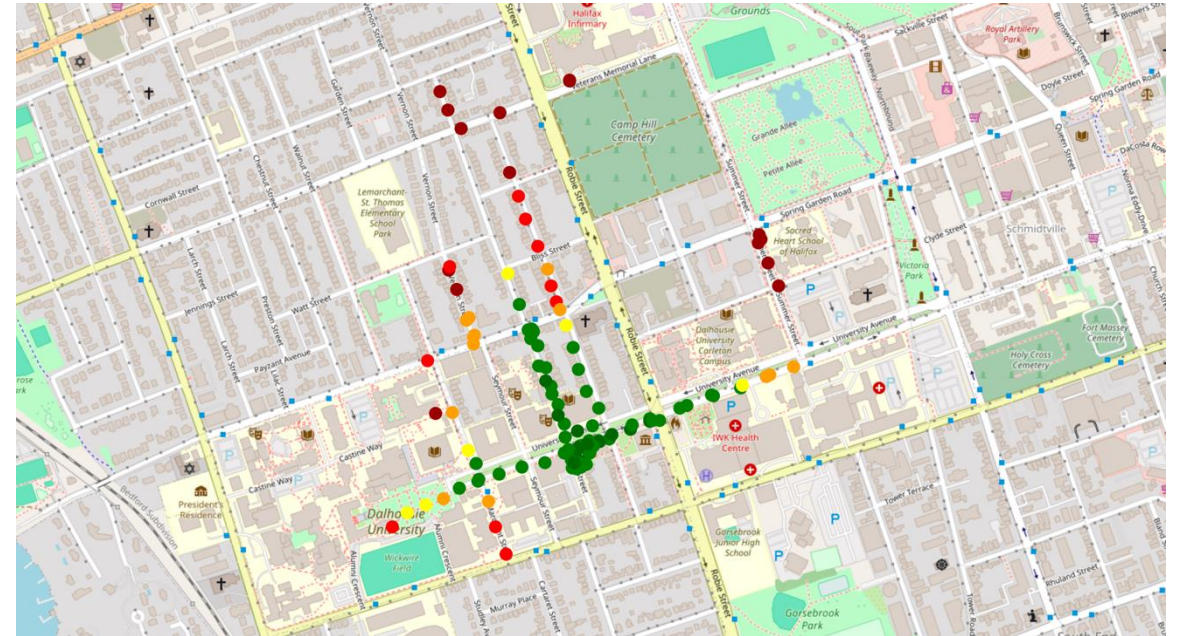
Points gathered between 3 routes

RSSI



- $\text{RSSI} > -90 \text{ dBm} = \text{Green}$
- $-110 \text{ dBm} < \text{RSSI} \leq -90 \text{ dBm} = \text{Yellow}$
- $-130 \text{ dBm} < \text{RSSI} \leq -110 \text{ dBm} = \text{Orange}$
- $\text{RSSI} \leq -130 \text{ dBm} = \text{Red}$

SNR

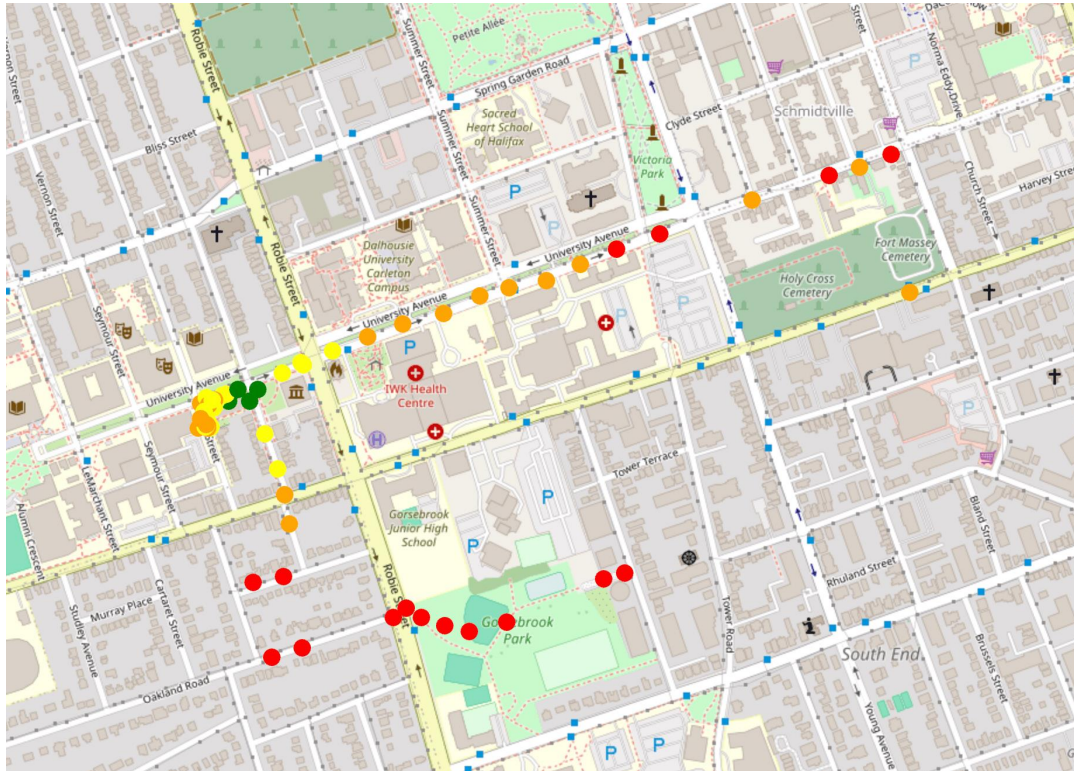


- $\text{SNR} > 10 \text{ dB} = \text{Dark Green}$
- $5 \text{ dB} < \text{SNR} \leq 10 \text{ dB} = \text{Green}$
- $0 \text{ dB} < \text{SNR} \leq 5 \text{ dB} = \text{Yellow}$
- $-5 \text{ dB} < \text{SNR} \leq 0 \text{ dB} = \text{Orange}$
- $-10 \text{ dB} < \text{SNR} \leq -5 \text{ dB} = \text{Red}$
- $\text{SNR} \leq -10 \text{ dB} = \text{Dark Red}$

LONG_FAST (Staircase)

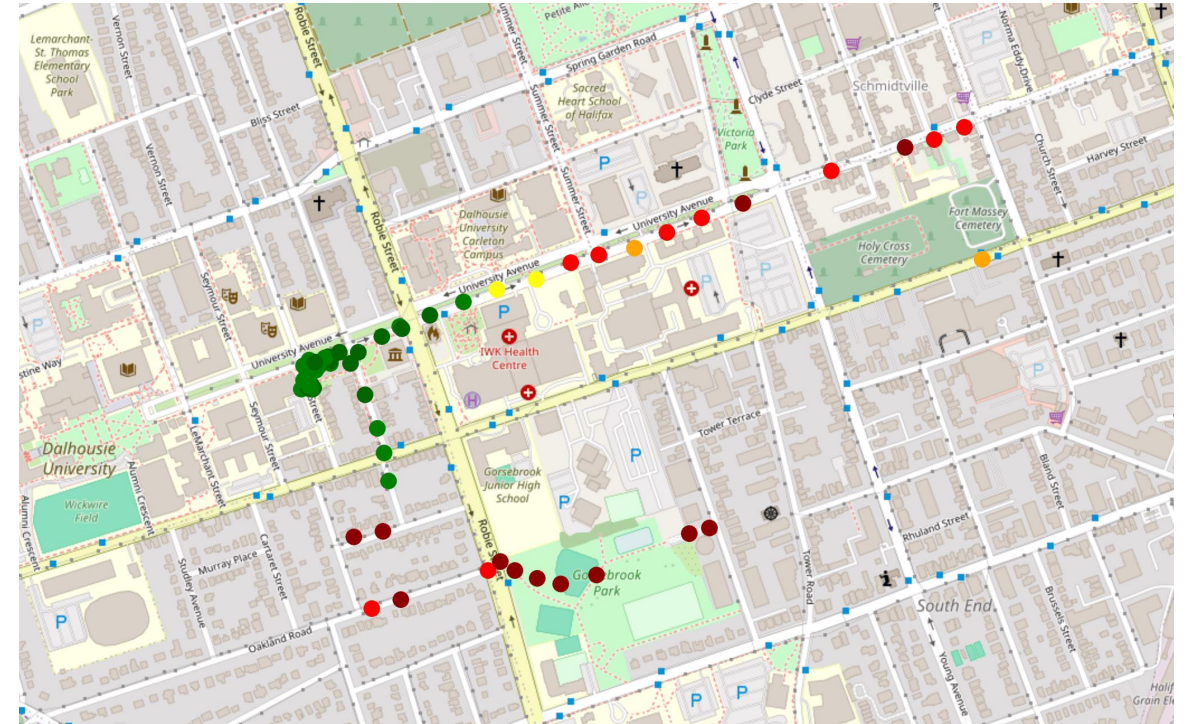
Points gathered between 2 routes

RSSI



- RSSI > -90 dBm = Green
- -110 dBm < RSSI ≤ -90 dBm = Yellow
- -130 dBm < RSSI ≤ -110 dBm = Orange
- RSSI ≤ -130 dBm = Red

SNR

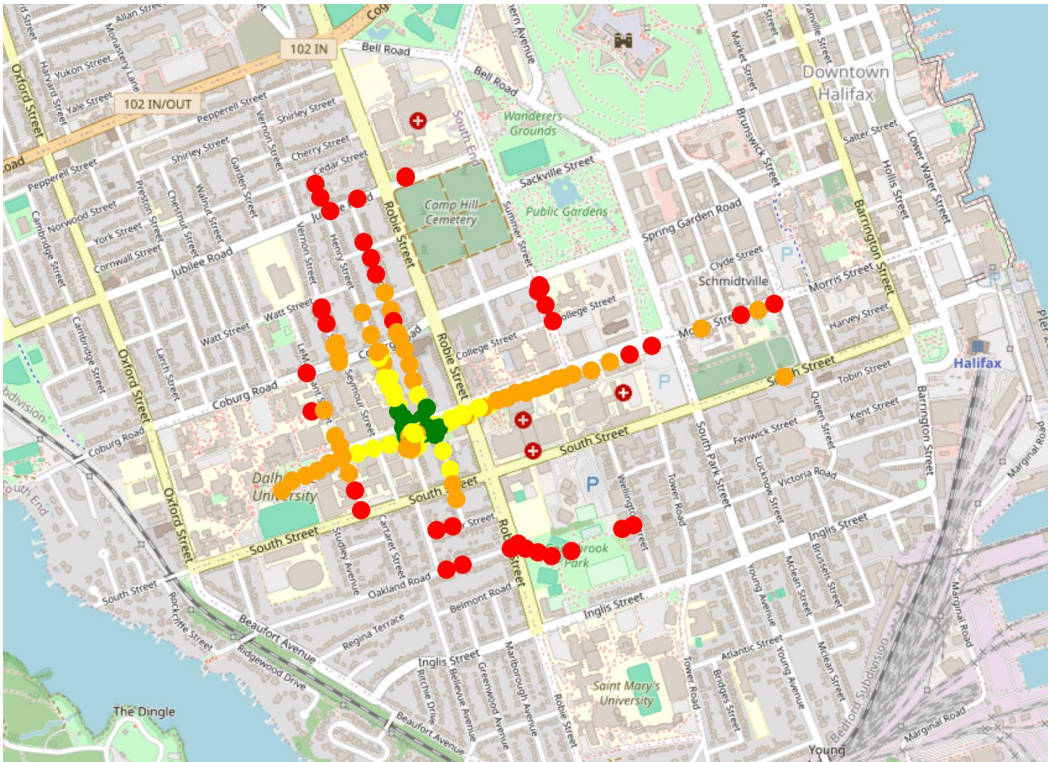


- SNR > 10 dB = Dark Green
- 5 dB < SNR ≤ 10 dB = Green
- 0 dB < SNR ≤ 5 dB = Yellow
- -5 dB < SNR ≤ 0 dB = Orange
- -10 dB < SNR ≤ -5 dB = Red
- SNR ≤ -10 dB = Dark Red

LONG_FAST (Both locations)

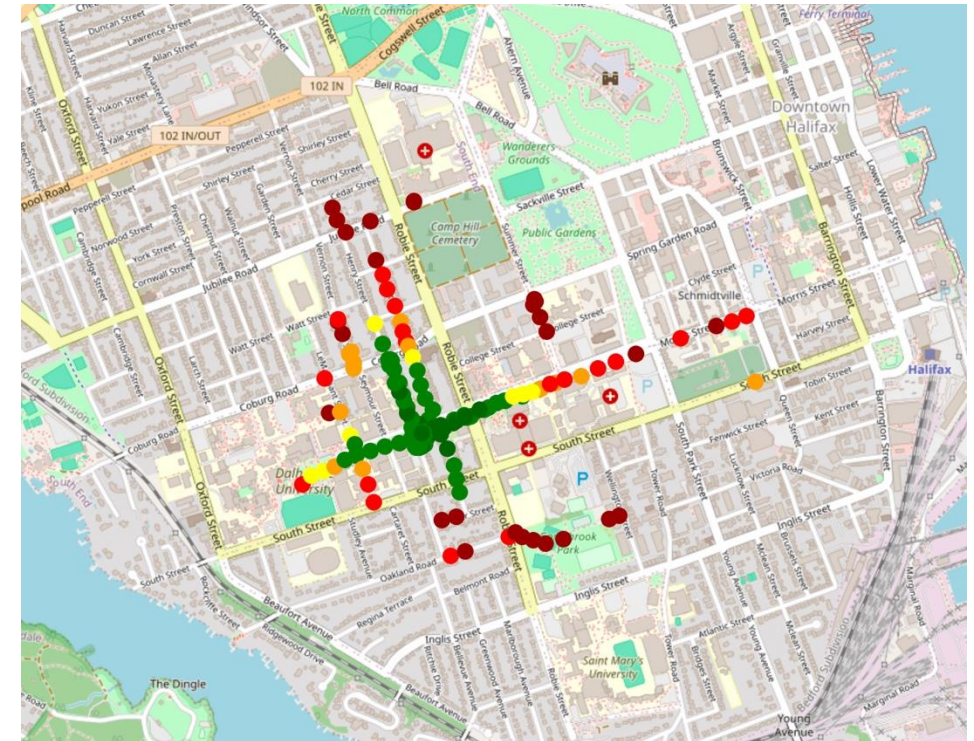
Points gathered between 5 routes

RSSI



- $\text{RSSI} > -90 \text{ dBm}$ = Green
- $-110 \text{ dBm} < \text{RSSI} \leq -90 \text{ dBm}$ = Yellow
- $-130 \text{ dBm} < \text{RSSI} \leq -110 \text{ dBm}$ = Orange
- $\text{RSSI} \leq -130 \text{ dBm}$ = Red

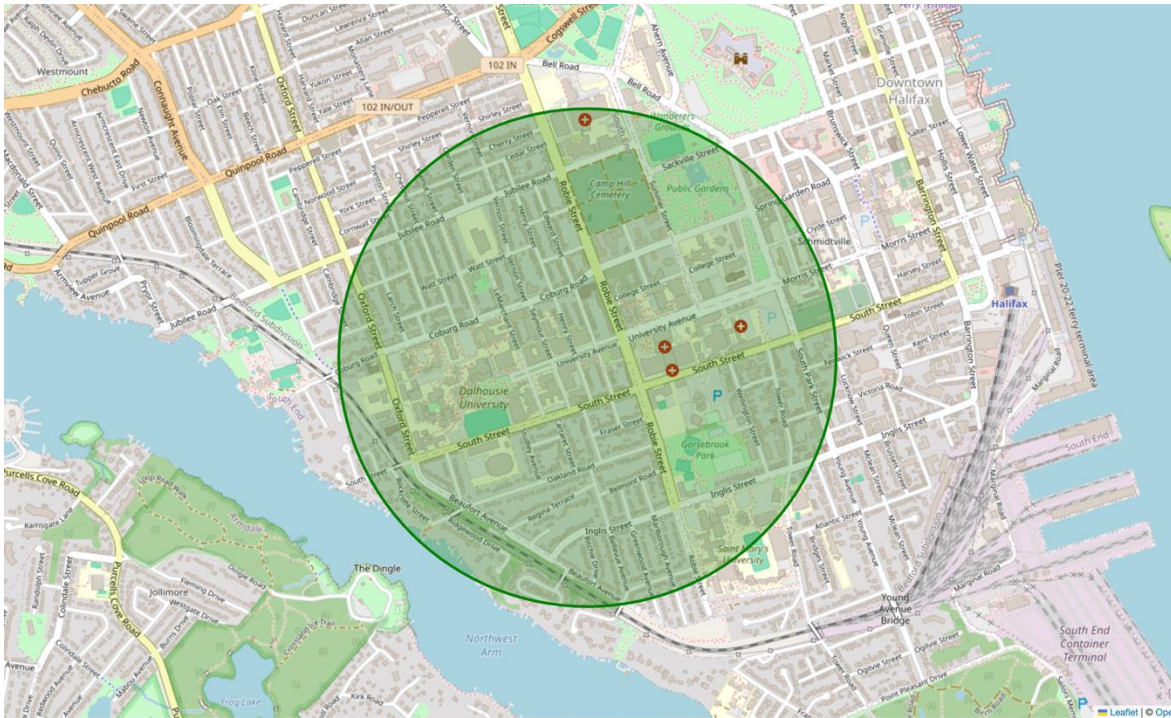
SNR



- $\text{SNR} > 10 \text{ dB}$ = Dark Green
- $5 \text{ dB} < \text{SNR} \leq 10 \text{ dB}$ = Green
- $0 \text{ dB} < \text{SNR} \leq 5 \text{ dB}$ = Yellow
- $-5 \text{ dB} < \text{SNR} \leq 0 \text{ dB}$ = Orange
- $-10 \text{ dB} < \text{SNR} \leq -5 \text{ dB}$ = Red
- $\text{SNR} \leq -10 \text{ dB}$ = Dark Red

LONG_FAST Approximate Range

Down an adjacent street (~900m)



With buildings (~500m)



Conclusion

- Increasing the transceiver's sensitivity helps immensely in an urban environment. The switch from SHORT_TURBO to LONG_FAST increased range 2.5-3x.
 - -117 dBm SHORT_TURBO to -133 dBm LONG_FAST.
- To maximize the efficiency of a LoRa mesh network in an urban environment like Halifax, it is vital to position nodes such that they take advantage of nearby streets.
 - Streets act as propagation corridors, having less obstacles and providing partial line-of-sight.
 - Tactically placing nodes in buildings on adjacent or nearby streets can improve range nearly up to 2x.

Next Week

- Deploy a small-scale mesh around campus based on range data gathered in the previous tests
- Gather data on the mesh including RSSI, SNR, delivery rate and hops
- Test both Meshtastic and MeshCore firmware on the same deployment, comparing delivery rate and efficiency (number of hops)
 - Determine key placement for 1-2 receiver nodes for MeshCore
- Doing so may require additional hardware, including LoRa transceivers and batteries.